JSOF

Vulnerability Basics

Prepared for [redacted]

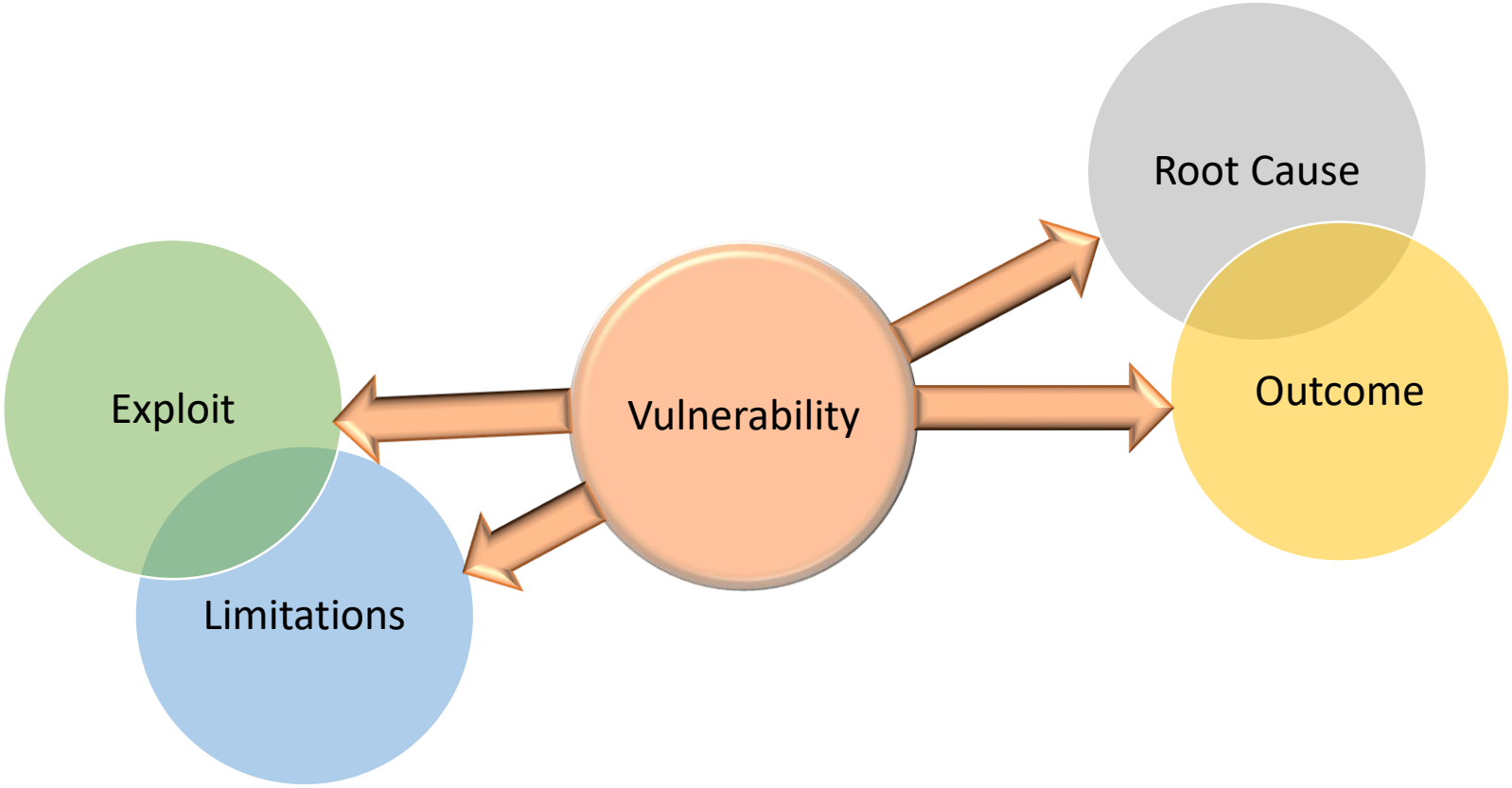
What is a vulnerability

Vulnerability is a software bug causing an information security problem

Developer makes a mistake → ships software → people and businesses are at risk

Can talk about “vulnerability” in other aspects, but we are focusing on software

What is a vulnerability



What is an exploit?

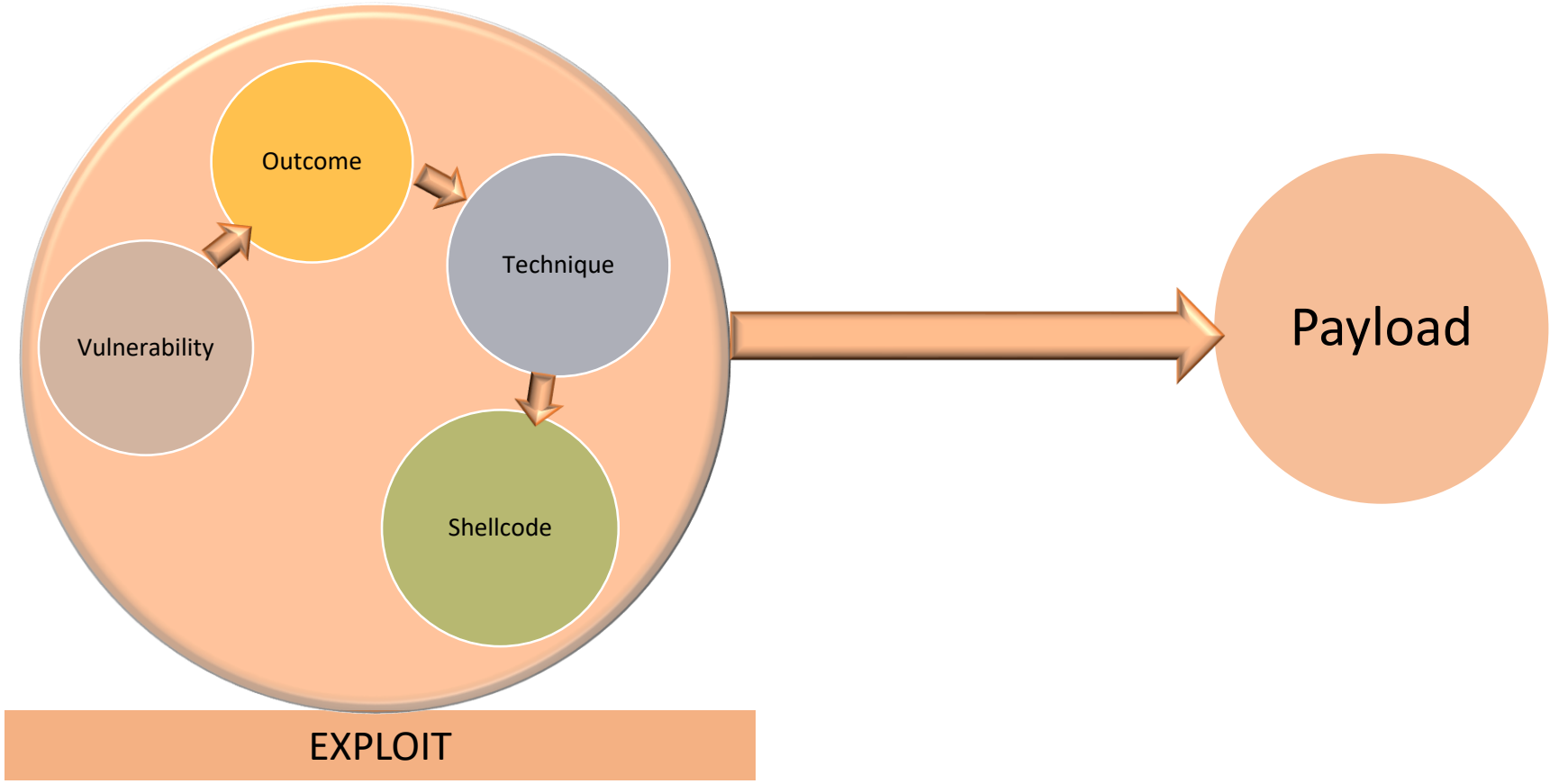
An exploit is something that an “attacker” writes

It turns a vulnerability in to an attack (“exploit a vulnerability”)

An exploit is usually suitable for one specific vulnerability and one specific device version

Very complex and many technique exploitation

What is an Exploit



What are exploit mitigations

It is very hard to avoid vulnerabilities

Because programmers are human, software is complex and many programming languages are not memory safe

Therefore it has been proven efficient to block exploits instead of vulnerabilities.

JSOF

What are exploit mitigations

- Exploit mitigations try to stop attackers from performing exploits
- Mitigations target exploit techniques and are implemented in Operating Systems, Compilers, directly in software and in hardware
- Cat-and-mouse game
- But very effective

Types of vulnerabilities

- Any programming or design bug in secure system can become a vulnerability
- but there are patterns.. And we categorize vulnerabilities in to types
- There are many different types of vulnerabilities

Many types of vulnerabilities

- Memory corruption bugs (we will come back to these..)
- Double-fetch/TOCTOU
- Backdoors
- Logic bugs
- Crypto logic bugs
- Architectural vulnerabilities (Spectre/Meltdown for example)

Many types of vulnerabilities

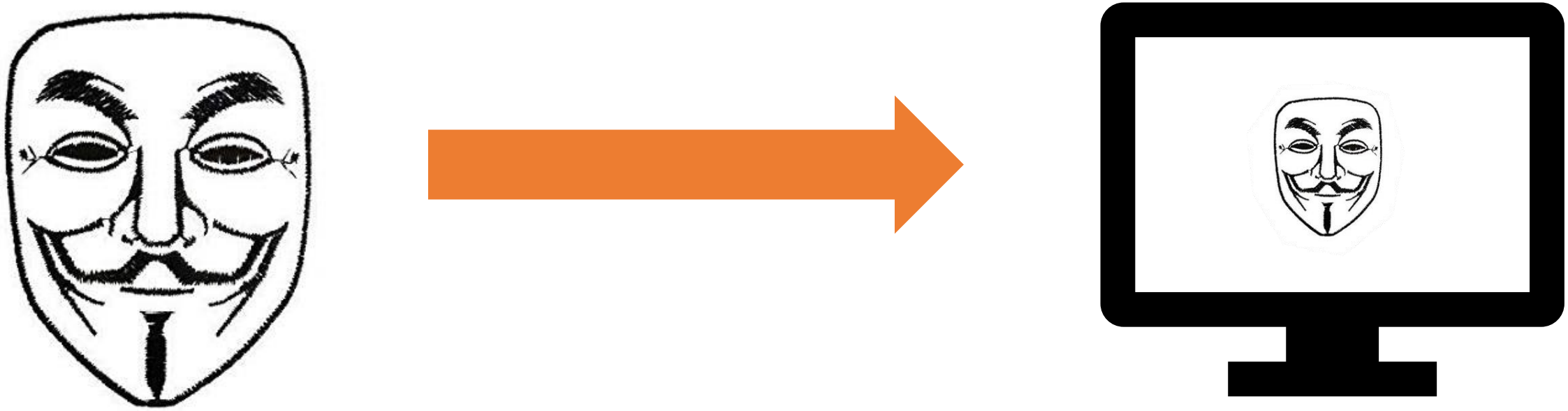
- Side channel bugs
- Format string bugs
- Type confusion
- Use-After-free
- Web vulnerabilities
- Database vulnerabilities
- And others..

We are focusing on “low level” vulnerabilities
We are focusing on “software” vulnerabilities

Many Outcomes for Vulnerabilities..

- Remote Code Execution
- EoP
- Information Leakage
- Authentication bypass or data access
- Reboot Persistency
- Denial of Service

Remote Code Execution



Remote Code Execution

- The worst/best type of outcome
- Allows an attacker to remotely execute code on a machine or device without authorization
- Often circumvented by limited privileges
- Abbreviated “RCE”

Elevation of Privilege

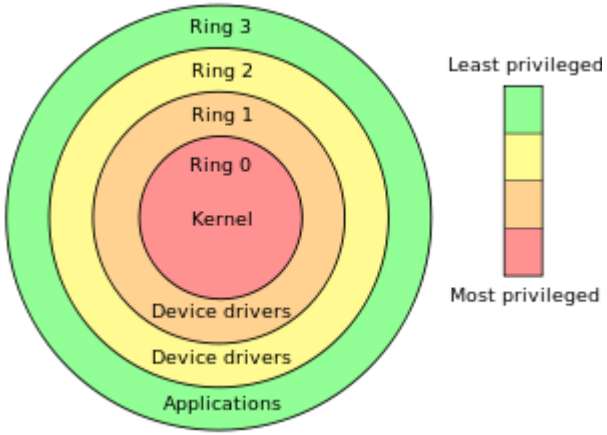
User	Admin	Root	Kernel	Hypervisor	SMM

Elevation of Privilege

- Allows an attacker to gain unauthorized privileges once already authorized or running code on a device

- Often a necessary stage since RCE will give us very low privileges or we already have some access..

- Abbreviated “EoP” or in some cases “PE”



Information Leakage

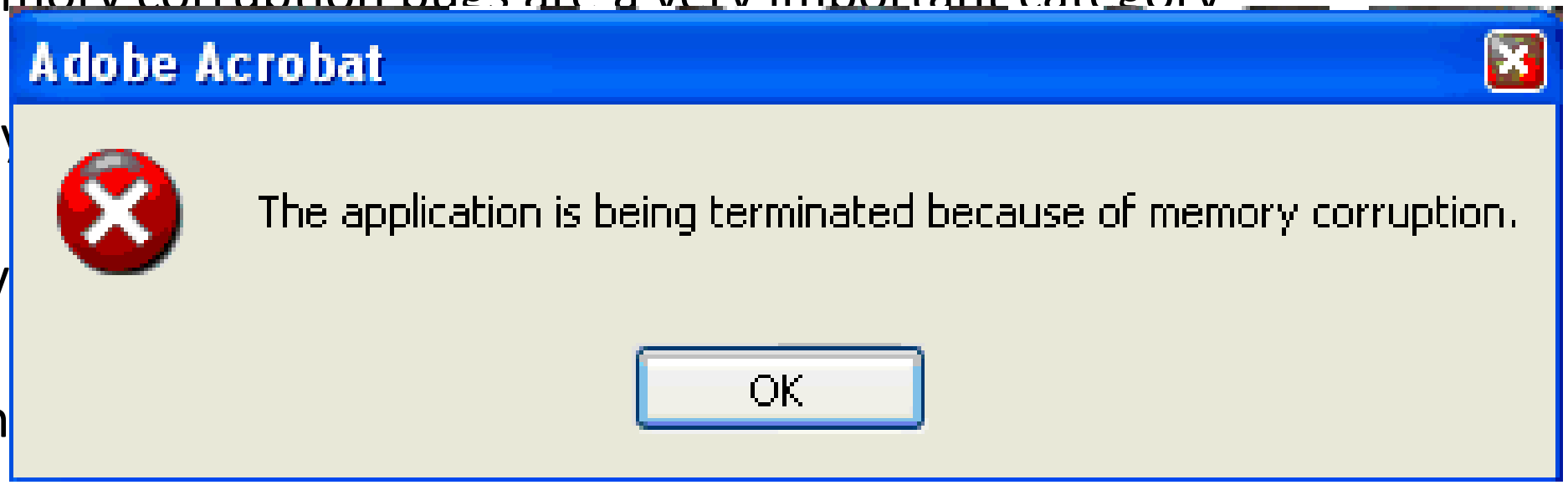
- Allows an attacker to gain information without authorization
- In some cases this will be secret information
- In most cases this will be information that is only protected to mitigate against other attacks.
  - For example: Memory addresses
- Very often an attacker will need an information leakage to perform an RCE or EoP

Memory corruption bugs

- Memory corruption bugs are a very important category
- Very common
- Have bad consequences
- Can be found and exploited via semi-repeatable techniques
- Mostly relevant for non-memory-safe languages
  - Or unsafe extensions
  - Or unsafe uses

Memory corruption bugs

- Memory corruption bugs are a very important category
- Very
- Hav
- Can



- Mostly relevant for non-memory-safe languages
  - Or unsafe extensions
  - Or unsafe uses

Types of memory corruption bugs

- Buffer Overflow
- Use-after-free
- Double-free
- Uninitialized memory
- Others..

Sample vulnerability types

Buffer Overflow

- A buffer is a allocated in a certain size
- And too much is written
- Can happen in the
  - Heap
  - Stack
  - Data
  - Other

Buffer Overflow

```
void buffer_overflow (char * src_buffer)
{
        char buffer_to_overrun[10];
        memcpy(buffer_to_overrun,  src_buffer, 13);
}
```

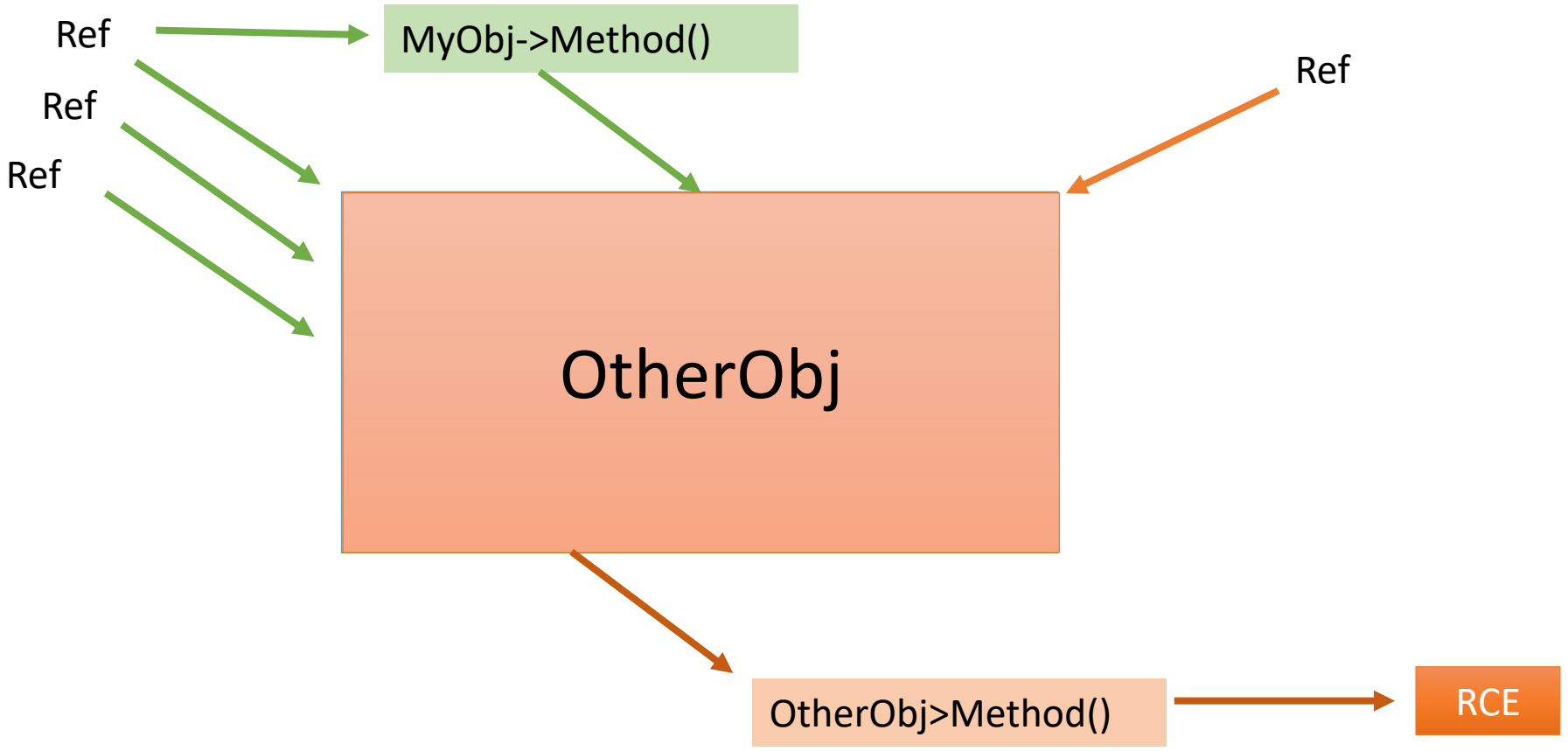
Use-After-Free

- A memory buffer is freed
- But not all references to the memory are cleared
- Then memory is reused
- And then the reference is used as if the data was never freed
  - Often as an object pointer..

Use-After-Free

- More common in CPP code
  - Because of complex logic and implicit (de)allocations
- More Common then in the past
- Very common in browsers
- Abbreviated UaF

Use-After-Free



TOCTOU

- “Time of check – Time of use”
- Occurs when a check is performed on data
- And then the data is re-fetched from the source
- And the source is changeable – for example a file or network resource
- In some cases called “double-fetch”

TOCTOU

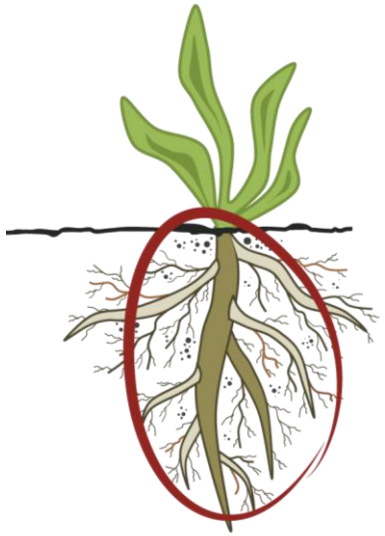
- “Time of chec
- Occurs when
- And then the
- And the sourc twork resource
- In some cases called “double-fetch”

TOCTOU

```
int toctou(int src_fd)
{
        char buffer_to_overflow[BUFFER_SIZE];
        unsigned int length = 0;
        read(src_fd, &length, sizeof(length));
        if (length > BUFFER_SIZE) return -1;
        read(src_fd, &length, sizeof(length));
        read(src_fd, buffer_to_overflow, length);

        return 0;
}
```

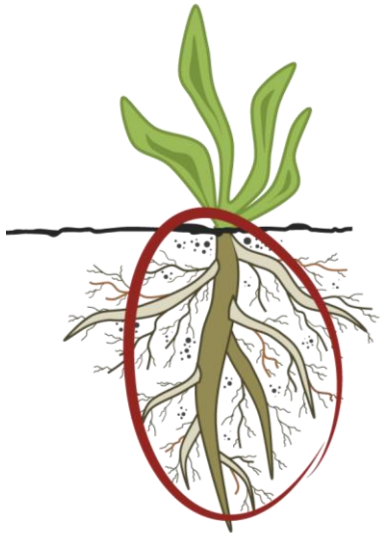
Vulnerability Root Causes



- Vulnerabilities are always caused by bugs
- Mostly in code
  - But sometimes in configuration or compilation(!)
- The C language is complex and C issues are a cause of many of the vulnerability root causes..
  - And a lot of code is written in C
  - Embedded in particular

Vulnerability Root Causes

- Arithmetic overflows (“integer overflow”)
- Signed/unsigned mismatch
- Truncation
- Off-by-one
- Operator precedence
- Undefined behavior
  - This can get complex



Undefined behavior

```
int undefined_behavior()
{
  int *p = (int*)malloc(sizeof(int));
  int *q = (int*)realloc(p, sizeof(int));
  *p = 1;
  *q = 2;
  if (p == q)
    printf("%d %d\n", *p, *q);
}
//code by Nick Lewycky
```

```
$ clang -O realloc.c ; ./a.out
1 2
```

Vulnerability life-cycle

Vulnerability users

- Vulnerabilities are found by different interest holders
  - Vendor internally
  - Security companies for PR
  - Bounty hunters
  - Security researchers as side-effect
  - Security researchers for competitions or bug bounty
  - Security researchers for profit
  - Governments
  - Criminals
  - In-the-wild while exploited

Vulnerability disclosure

- If a vulnerability is found and is not going to be used it is usually reported
- “Responsible disclosure” – give the vendor X number of days and then release to the public
- Vendor should give researcher credit for finding the bug
- Vendor will often report a vulnerability even if found internally

Vulnerability disclosure

- A bad vendor will:
  - Threaten to sue security researcher
  - Deny vulnerability
  - Not report vulnerability

- A good vendor will:
  - Respond quickly
  - Give the reporter credit
  - Remediate the vulnerability, provide update and tell users or customers about the fix

Vulnerability

- A bad vendor wil
  - Threaten to sue
  - Deny vulnerabili
  - Not report vulne
- A good vendor w
  - Respond quickly
  - Give the reporte
  - Remediate the v ell users or customers about the fix

- CVE-2017-13220

  Al Viro reported that the Bluetooth HID this to cause a denial of service.

- CVE-2017-16526

  Andrey Konovalov reported that the UV service.

- CVE-2017-16911

  Secunia Research reported that the US other vulnerabilities.

- CVE-2017-16912

  Secunia Research reported that the US A remote user able to connect to the U

Bug bounties

- Many vendors have bug bounties
- They are mostly about fame, not money

CO

-
-
-

Mobile Pwn2Own™

Vulnerability monitoring

Base Score

Attack Vector (AV)

Network (N) Adjacent (A)

Attack Complexity (AC)

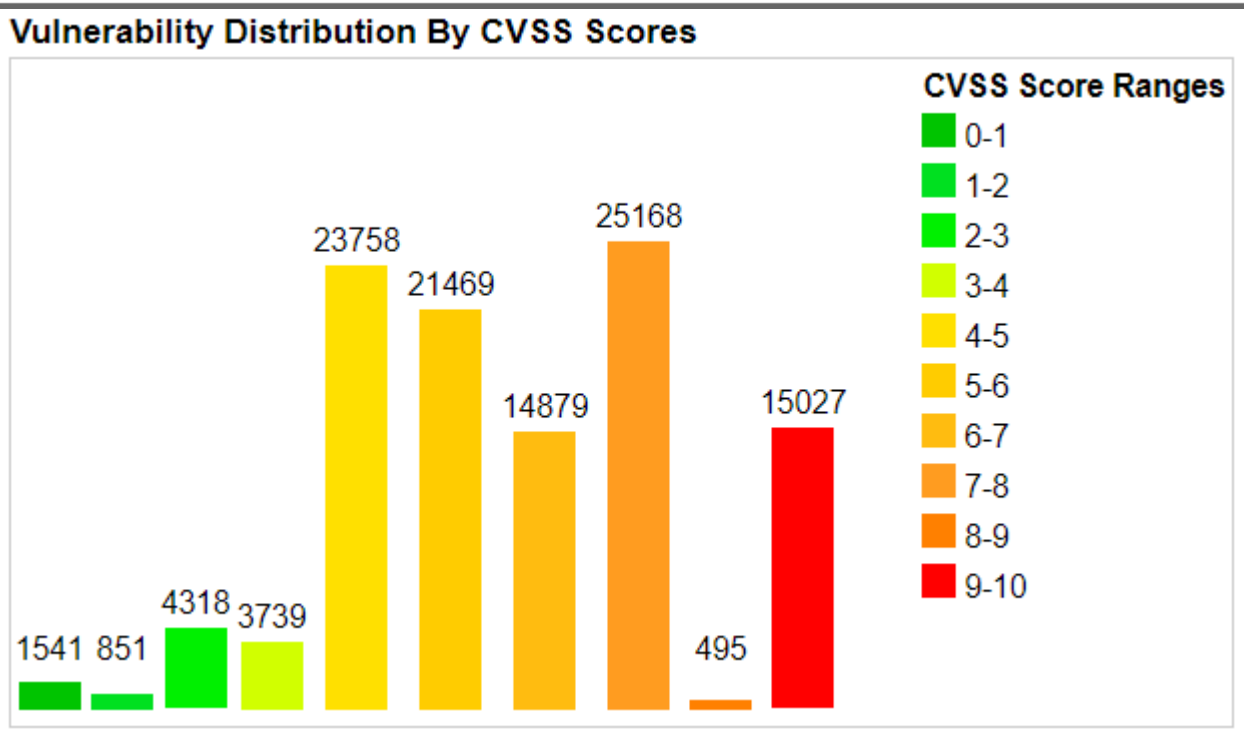
Low (L) High (H)

Privileges Required (PR)

None (N) Low (L) High (

User Interaction (UI)

None (N) Required (R)



anged (C)

High (H)

High (H)

High (H)

Vector String - select values for all base metrics to generate a vector

Vulnerability regulation

- Many countries regulate vulnerability exports
- In Europe there are “Wassenar” rules

- Israel follows Europe +-
- Short summary – you can freely publish or give to vendor, any other export needs authorization
  - This means in theory you can’t participate in some competitions..
  - Including products which use exploits
- 1-days are OK

What to read/watch

- Big conferences
  - Black Hat (multiple) & Defcon
  - Hat in the box (multiple)
  - Recon
  - Infiltrate
  - CCC
- Academic
  - arxiv.org -> security
  - Ieee -> security
  - Usenix -> security

What to read/watch

- Conferences in Israel!
  - DC9723
  - BluehatIL
  - BsidesTLV
  - Cyberweek TLV (Tel-Aviv university)
- Twitter
  - Many many names to follow…
    - Start from some GOOF, MSFT and TENCENT researchers and follow from there…
- Blogs
  - Better to follow content from twitter

What to read/watch

- Books
  - The art of software security assessment
  - The Shellcoder’s handbook
  - Hacking: The Art of Exploitation
  - Gray Hat Hacking
  - Specific books for target or topics

Let’s Practice